

This project fine-tunes a pre-trained BERT model with an additional LSTM layer for the task of sentiment classification (positive/negative) using the SST-2 dataset.

```
!pip install torch torchvision torchaudio transformers scikit-learn datasets evaluate
```

Requirement already satisfied: evaluate in /usr/local/lib/python3.11/dist-packages (0.4.3)

Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch) (3.18.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch) (4.13.2)

Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch) (3.4.2)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/dist-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)

Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch) (9.1.0.70)

Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.5.8)

Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in /usr/local/lib/python3.11/dist-packages (from torch) (11.2.1.3)

Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in /usr/local/lib/python3.11/dist-packages (from torch) (10.3.5.147)

Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in /usr/local/lib/python3.11/dist-packages (from torch) (11.6.1.9)

Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in /usr/local/lib/python3.11/dist-packages (from torch) (12.3.1.170)

Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in /usr/local/lib/python3.11/dist-packages (from torch) (0.6.2)

Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from torch) (2.21.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)

Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-packages (from torch) (3.2.0)

Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch) (1.13.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch) (1.3.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from torchvision) (2.0.2)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.11/dist-packages (from torchvision) (11.2.1)

Requirement already satisfied: huggingface-hub<1.0,>=0.30.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.30.2)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (24.2)

Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)

Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.11.6)

Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)

Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.1)

Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.5.3)

Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.11/dist-packages (from transformers) (4.67.1)

Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.15.2)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (18.1.0)

Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.3.8)

Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from datasets) (2.2.2)

Requirement already satisfied: xxhash in /usr/local/lib/python3.11/dist-packages (from datasets) (3.5.0)

Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.11/dist-packages (from datasets) (0.70.16)

Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-packages (from datasets) (3.11.15)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (2.6.1)

Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.3.2)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (25.3.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.6.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (6.4.3)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (0.3.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from aiohttp->datasets) (1.20.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2.4.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->transformers) (2025.1.31)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-packages (from Jinja2->torch) (3.0.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1

Install libraries

```
import numpy as np
from datasets import load_dataset

from transformers import AutoTokenizer, DataCollatorWithPadding, AutoModelForSequenceClassification, TrainingArguments, Trainer, AutoModel

import torch
from torch.utils.data import DataLoader
import torch.nn as nn
from torch.optim import AdamW
from torch.optim.lr_scheduler import ReduceLROnPlateau

from sklearn.metrics import accuracy_score, f1_score
```

```
raw_dataset = load_dataset("glue", "sst2")
raw_dataset
```

```
DatasetDict({
  train: Dataset({
    features: ['sentence', 'label', 'idx'],
    num_rows: 67349
  })
  validation: Dataset({
    features: ['sentence', 'label', 'idx'],
    num_rows: 872
  })
  test: Dataset({
    features: ['sentence', 'label', 'idx'],
    num_rows: 1821
  })
})
```

```
train_ds = raw_dataset["train"]
train_ds[10]
```

```
{'sentence': 'goes to absurd lengths ', 'label': 0, 'idx': 10}
```

```
train_ds.features
```

```
{'sentence': Value(dtype='string', id=None),
 'label': ClassLabel(names=['negative', 'positive'], id=None),
 'idx': Value(dtype='int32', id=None)}
```

```
checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```

```
def tokenize_data(data):
    res = tokenizer(data["sentence"], truncation=True, padding=False, max_length=None)
    res['labels'] = data['label']
    return res
```

```
tokenized_datasets = raw_dataset.map(tokenize_data, batched=True, remove_columns=["sentence", "label"])
```

```
Map: 100% 872/872 [00:02<00:00, 361.69 examples/s]
```

```
tokenized_datasets
```

```
DatasetDict({
  train: Dataset({
    features: ['idx', 'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
    num_rows: 67349
  })
  validation: Dataset({
    features: ['idx', 'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
    num_rows: 872
  })
  test: Dataset({
    features: ['idx', 'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
    num_rows: 1821
  })
})
```

```
# Dynamic Padding for train, validation and test datasets
data_collator = DataCollatorWithPadding(tokenizer=tokenizer, padding=True, return_tensors = "pt")
```

```
train_dataloader = DataLoader(
    tokenized_datasets["train"],
    shuffle = True,
    batch_size = 32,
    collate_fn = data_collator,
)
```

```
val_dataloader = DataLoader(
    tokenized_datasets["validation"],
    shuffle = True,
    batch_size = 32,
    collate_fn = data_collator,
)
```

```
test_dataloader = DataLoader(
    tokenized_datasets["test"],
    shuffle = True,
    batch_size = 32,
    collate_fn = data_collator,
)
```

```
train_ds[5675]
```

```
↩ {'sentence': 'no unifying rhythm or visual style ', 'label': 0, 'idx': 5675}
```

```
# Defining the BERT+LSTM model
```

```
class BERT_LSTM(nn.Module):
    def __init__(self, bert_model_name = checkpoint, hidden_dim=256, num_classes=2, dropout_rate=0.3):
        super(BERT_LSTM, self).__init__()
        self.bert = AutoModel.from_pretrained(bert_model_name)
        self.dropout1 = nn.Dropout(dropout_rate) # Add dropout after BERT for regularization
```

```
    #Bidirectional LSTM
    self.lstm = nn.LSTM(
        input_size = self.bert.config.hidden_size,
        hidden_size=hidden_dim,
        batch_first=True,
        bidirectional=True,
        dropout=dropout_rate if hidden_dim > 1 else 0)
```

```
    self.dropout2 = nn.Dropout(dropout_rate) # Additional dropout before classification
```

```
    self.classifier = nn.Linear(hidden_dim*2, num_classes)
```

```
    def forward(self, input_ids, attention_mask):
        # Passing inputs through BERT
        outputs = self.bert(input_ids = input_ids, attention_mask = attention_mask)
        sequence_output = outputs[0]
```

```
        # Apply dropout after BERT
        sequence_output = self.dropout1(sequence_output)
```

```
        # Pass BERT output through LSTM head
        lstm_out, _ = self.lstm(sequence_output)
```

```
        # Get output from last token and apply dropout
        final_hidden = self.dropout2(lstm_out[:, -1, :])
```

```
        # Get output from last token
        output = self.classifier(final_hidden)
```

```
    return output
```

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
```

```
↩ 2.6.0+cu124
   True
```

```
# Setting up the training parameters
```

```
# Initializing the model, optimizer and loss function
model = BERT_LSTM()
optimizer = AdamW(model.parameters(), lr=2e-5)
loss_fn = nn.CrossEntropyLoss()
```

```
# Moving the model to available GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print
model.to(device)
```

```
↩ /usr/local/lib/python3.11/dist-packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout after all but last recurrence
  warnings.warn(
BERT_LSTM(
  (bert): BertModel(
```

```

(embeddings): BertEmbeddings(
  (word_embeddings): Embedding(30522, 768, padding_idx=0)
  (position_embeddings): Embedding(512, 768)
  (token_type_embeddings): Embedding(2, 768)
  (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
  (dropout): Dropout(p=0.1, inplace=False)
)
(encoder): BertEncoder(
  (layer): ModuleList(
    (0-11): 12 x BertLayer(
      (attention): BertAttention(
        (self): BertSdpaSelfAttention(
          (query): Linear(in_features=768, out_features=768, bias=True)
          (key): Linear(in_features=768, out_features=768, bias=True)
          (value): Linear(in_features=768, out_features=768, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (output): BertSelfOutput(
          (dense): Linear(in_features=768, out_features=768, bias=True)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
      (intermediate): BertIntermediate(
        (dense): Linear(in_features=768, out_features=3072, bias=True)
        (intermediate_act_fn): GELUActivation()
      )
      (output): BertOutput(
        (dense): Linear(in_features=3072, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (activation): Tanh()
  )
)
(dropout1): Dropout(p=0.3, inplace=False)
(lstm): LSTM(768, 256, batch_first=True, dropout=0.3, bidirectional=True)
(dropout2): Dropout(p=0.3, inplace=False)
(classifier): Linear(in_features=512, out_features=2, bias=True)
)

```

```
from tqdm.notebook import tqdm
```

```
from torch.amp import autocast, GradScaler
```

```
# Training Loop
```

```
scaler = GradScaler()
```

```
def train(model, train_dataloader, optimizer, loss_fn, device):
```

```
    model.train()
```

```
    total_loss=0
```

```
    all_preds=[]
```

```
    all_labels=[]
```

```
    for batch in tqdm(train_dataloader):
```

```
        input_ids = batch['input_ids'].to(device)
```

```
        attention_mask = batch['attention_mask'].to(device)
```

```
        labels = batch['labels'].to(device)
```

```
    # Zero out the gradients
```

```
    optimizer.zero_grad()
```

```
    with autocast(device_type='cuda'):
```

```
        # Perform forward pass and compute loss
```

```
        outputs = model(input_ids, attention_mask)
```

```
        logits = outputs
```

```
        preds = torch.argmax(logits, dim=1)
```

```
        # preds = (logits > 0.5).long()
```

```
        loss = loss_fn(logits.squeeze(), labels)
```

```
    # Perform Backpropogation
```

```
    scaler.scale(loss).backward()
```

```
    scaler.step(optimizer)
```

```
    scaler.update()
```

```

# Track metrics
total_loss += loss.item()

# labels = torch.argmax(labels, dim=1)

all_preds.extend(preds.cpu().detach().numpy())
all_labels.extend(labels.cpu().detach().numpy())

all_preds = np.array(all_preds, dtype=int)
all_labels = np.array(all_labels, dtype=int)
avg_loss = total_loss / len(train_dataloader)
accuracy = accuracy_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds, average='weighted')

return avg_loss, accuracy, f1

# Validation Loop to track overfitting
def evaluate(model, val_dataloader, loss_fn, device):
    model.eval()
    total_loss = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for batch in tqdm(val_dataloader):
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            with autocast(device_type='cuda'):
                outputs = model(input_ids, attention_mask)
                logits = outputs
                preds = torch.argmax(logits, dim=1)
                # loss = loss_fn(logits, labels)

            # preds = (logits > 0.5).long()
            loss = loss_fn(logits.squeeze(), labels)

            total_loss += loss.item()

            # labels = torch.argmax(labels, dim=1)
            all_preds.extend(preds.cpu().detach().numpy())
            all_labels.extend(labels.cpu().detach().numpy())

    all_preds = np.array(all_preds, dtype=int)
    all_labels = np.array(all_labels, dtype=int)

    avg_loss = total_loss / len(val_dataloader)
    accuracy = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds, average='weighted')
    return avg_loss, accuracy, f1

class EarlyStopping:
    def __init__(self, patience=3, min_delta=0.005, verbose=True):
        self.patience = patience
        self.min_delta = min_delta
        self.verbose = verbose
        self.counter = 0
        self.best_score = None
        self.early_stop = False
        self.val_loss_min = np.inf

    def __call__(self, val_loss, model, path='best_model.pt'):
        score = -val_loss

        if self.best_score is None:
            self.best_score = score
            self.save_checkpoint(val_loss, model, path)
        elif score < self.best_score + self.min_delta:
            self.counter += 1
            if self.verbose:
                print(f'Early stopping counter: {self.counter} out of {self.patience}')
            if self.counter >= self.patience:
                self.early_stop = True

```

```

        else:
            self.best_score = score
            self.save_checkpoint(val_loss, model, path)
            self.counter = 0

    def save_checkpoint(self, val_loss, model, path):
        if self.verbose:
            print(f'Validation loss decreased ({self.val_loss_min:.6f} --> {val_loss:.6f}). Saving model...')
        torch.save(model.state_dict(), path)
        self.val_loss_min = val_loss

# Setting up the training pipeline

def train_model(model, train_dataloader, val_dataloader, device, num_epochs=3, lr=2e-5):
    # Setup the parameters
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    loss_fn = nn.CrossEntropyLoss()

    # Early stopping
    early_stopping = EarlyStopping(patience=3, verbose=True)

    # Learning rate scheduler - updates based on validation loss
    scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2, verbose=True)

    # Training and validation history
    history = {
        'train_loss': [], 'train_acc': [], 'train_f1': [],
        'val_loss': [], 'val_acc': [], 'val_f1': []
    }

    # Main training loop
    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")

        # Training phase
        train_loss, train_acc, train_f1 = train(model, train_dataloader, optimizer, loss_fn, device)
        print(f"Train Loss: {train_loss:.3f} | Train Accuracy: {train_acc:.3f} | Train F1: {train_f1:.3f}")

        # Validation phase
        val_loss, val_acc, val_f1 = evaluate(model, val_dataloader, loss_fn, device)
        print(f"Validation Loss: {val_loss:.3f} | Validation Accuracy: {val_acc:.3f} | Validation F1: {val_f1:.3f}")

        # Update learning rate based on validation loss
        scheduler.step(val_loss)

        # Check early stopping
        early_stopping(val_loss, model)

        # Save history
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['train_f1'].append(train_f1)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)
        history['val_f1'].append(val_f1)

        if early_stopping.early_stop:
            print("Early stopping triggered!")
            break

    return model, history

# torch.save(model.state_dict(), "bert_lstm_sentiment.pth")
# from google.colab import files
# files.download("bert_lstm_sentiment.pth")

# Set training parameters
num_epochs = 3
learning_rate = 2e-5

# Call the train_model function
trained_model, history = train_model(
    model=model,

```

```
train_dataloader=train_dataloader,
val_dataloader=val_dataloader,
device=device,
num_epochs=num_epochs,
lr=learning_rate
)

# Visualize the training history
import matplotlib.pyplot as plt

# Setup figure and subplots
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 5))

# Plot Loss
epochs = range(1, len(history['train_loss']) + 1)
ax1.plot(epochs, history['train_loss'], 'b-', label='Training Loss')
ax1.plot(epochs, history['val_loss'], 'r-', label='Validation Loss')
ax1.set_title('Training & Validation Loss')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Loss')
ax1.legend()

# Plot Accuracy
ax2.plot(epochs, history['train_acc'], 'b-', label='Training Accuracy')
ax2.plot(epochs, history['val_acc'], 'r-', label='Validation Accuracy')
ax2.set_title('Training & Validation Accuracy')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Accuracy')
ax2.legend()

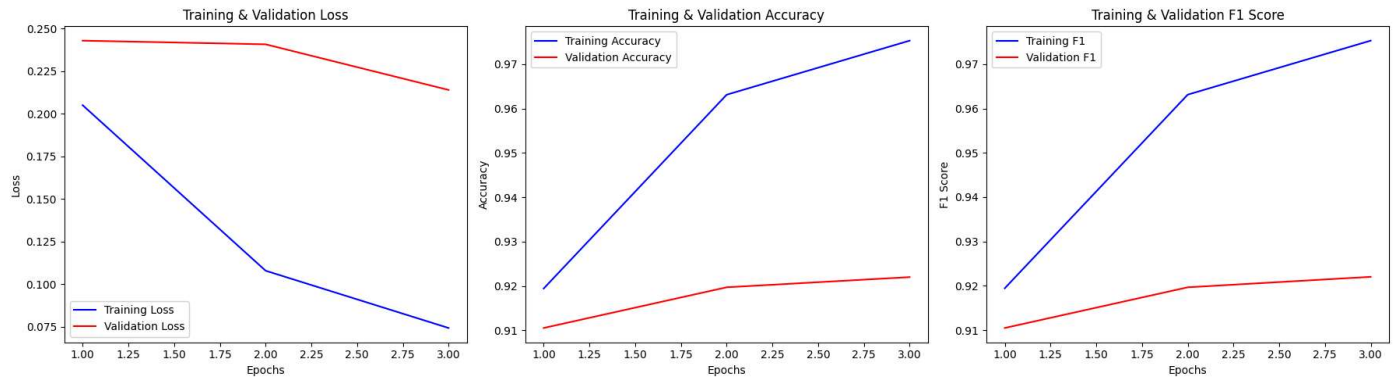
# Plot F1 Score
ax3.plot(epochs, history['train_f1'], 'b-', label='Training F1')
ax3.plot(epochs, history['val_f1'], 'r-', label='Validation F1')
ax3.set_title('Training & Validation F1 Score')
ax3.set_xlabel('Epochs')
ax3.set_ylabel('F1 Score')
ax3.legend()

plt.tight_layout()
plt.show()
```

```

Epoch 1/3
100% 2105/2105 [03:51<00:00, 9.71it/s]
Train Loss: 0.205 | Train Accuracy: 0.919 | Train F1: 0.919
100% 28/28 [00:00<00:00, 40.17it/s]
Validation Loss: 0.243 | Validation Accuracy: 0.911 | Validation F1: 0.910
Validation loss decreased (inf --> 0.242918). Saving model...
Epoch 2/3
100% 2105/2105 [03:42<00:00, 9.60it/s]
Train Loss: 0.108 | Train Accuracy: 0.963 | Train F1: 0.963
100% 28/28 [00:00<00:00, 40.98it/s]
Validation Loss: 0.241 | Validation Accuracy: 0.920 | Validation F1: 0.920
Early stopping counter: 1 out of 3
Epoch 3/3
100% 2105/2105 [03:41<00:00, 9.45it/s]
Train Loss: 0.074 | Train Accuracy: 0.975 | Train F1: 0.975
100% 28/28 [00:00<00:00, 41.10it/s]
Validation Loss: 0.214 | Validation Accuracy: 0.922 | Validation F1: 0.922
Validation loss decreased (0.242918 --> 0.213992). Saving model...

```



```

for i in range(num_epochs):
    print(f"Epoch {i+1}/{num_epochs}")
    print(f"Train loss: {history['train_loss'][i]}")
    print(f"Train accuracy: {history['train_acc'][i]}")
    print(f"Train F1 score: {history['train_f1'][i]}")
    print(f"Validation loss: {history['val_loss'][i]}")
    print(f"Validation accuracy: {history['val_acc'][i]}")
    print(f"Validation F1 score: {history['val_f1'][i]}")
    print('\n')

```

```

Epoch 1/3
Train loss: 0.20504201198983363
Train accuracy: 0.9194494350324429
Train F1 score: 0.9194428049667923
Validation loss: 0.24291771969624928
Validation accuracy: 0.9105504587155964
Validation F1 score: 0.9104784339816726

```

```

Epoch 2/3
Train loss: 0.10794312086602288
Train accuracy: 0.9631323404950334
Train F1 score: 0.9631379843808998
Validation loss: 0.24076091232044355
Validation accuracy: 0.9197247706422018
Validation F1 score: 0.919660062227382

```

```

Epoch 3/3
Train loss: 0.07433691809303404
Train accuracy: 0.9753225734606304
Train F1 score: 0.9753270626003193
Validation loss: 0.2139917659972395
Validation accuracy: 0.9220183486238532
Validation F1 score: 0.9220232722259605

```



```

# Since the sst2 test data does not specify label, we can only make predictions, and cannot calculate the metrics
all_preds = []
test_ds = raw_dataset["test"]

model.eval()
with torch.no_grad():
    for batch in test_dataloader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs
        preds = torch.argmax(logits, dim=-1)

        all_preds.extend(preds.cpu().numpy())

for i in range(5):
    rndm_idx = np.random.randint(0, len(test_ds), 1)
    print(f"Test index number:{rndm_idx}")
    print(f"Sentence: {test_ds[rndm_idx]['sentence']}")
    print(f"Prediction:{all_preds[rndm_idx.item()]}")
    print('\n')

↩ Test index number:[1501]
Sentence: ['an exciting and involving rock music doc , a smart and satisfying look inside that tumultuous world .']
Prediction:0

Test index number:[1576]
Sentence: ['everytime you think undercover brother has run out of steam , it finds a new way to surprise and amuse .']
Prediction:0

```